```
Detecting operating system of created instance...
Waiting for SSH to be available...
Detecting the provisioner...
Provisioning with boot2docker...
Copying certs to the local machine directory...
Copying certs to the remote machine...
Setting Docker configuration on the remote daemon...
Checking connection to Docker...
Docker is up and running!
To see how to connect your Docker Client to the Docker Engine running on
this virtual machine, run: docker-machine env local-vm-1
```

This command says, "Create a VirtualBox instance named local-vm-1." The output shows the various steps it goes through to provision our instance. You can see, for example, that it uses an image called boot2docker.iso; boot2docker is a lightweight Linux distro that's optimized for running Docker.

We can verify our instance has been created by listing the instances that docker-machine knows about:

```
$ docker-machine ls
NAME       ACTIVE DRIVER     STATE   URL       SWARM DOCKER      ERRORS
local-vm-1 -      virtualbox Running tcp://19…       v18.09.1
```

What can we do with our new instance? Let's take it for a little spin.

For a start, we can SSH onto it:

```
$ docker-machine ssh local-vm-1
   ( '>')
  /) TC (\   Core is distributed with ABSOLUTELY NO WARRANTY.
 (/-_--_-\)           www.tinycorelinux.net

docker@local-vm-1:~$
```

We can see that Docker is installed:

```
docker@local-vm-1:~$ docker -v
Docker version 18.09.1, build 4c52b90
```

Having SSH'd onto the machine, we could issue our Docker commands directly here to start Docker services on this new host.

Alternatively, if we exit this SSH session and return to our local machine:

```
docker@local-vm-1:~$ exit
```

we can then issue commands against this new instance by specifying a command string at the end of the SSH command:

```
$ docker-machine ssh <instance name> "<command>"
```